



南開大學
Nankai University

计算机学院

并行程序设计实验报告

n 个数求和的链式、指令级并行与树形归
约算法实验

姓名：马禹翔

学号：2410933

专业：计算机科学与技术

2026 年 3 月 28 日

目录

1	实验目的	2
2	实验原理	2
2.1	问题描述	2
2.2	平凡链式累加算法	3
2.3	两路链式累加与指令级并行	3
2.4	树形归约算法	3
2.5	并行树形归约	3
3	实验平台与实验方案	4
3.1	实验平台	4
3.2	测试数据设计	4
3.3	测试规模设计	4
3.4	计时方法	5
4	算法设计与代码实现	5
4.1	算法思路说明	5
4.2	核心代码	5
4.3	代码结构与关键实现解析	9
4.4	伪代码描述	11
5	实验结果	13
6	实验过程中遇到的问题与解决方法	14
6.1	树形归约结果出错	14
6.2	对“优化”概念的理解偏差	14
7	结论	15

代码仓库说明

本实验完整源码已上传至 GitHub，访问地址如下：

https://github.com/s1nk141/nku26/blob/create/lab1_2.zip

1 实验目的

本实验围绕“计算 n 个数之和”这一基础问题，比较不同算法结构在现代处理器上的性能表现，具体目标如下：

1. 理解平凡链式累加算法的计算过程及其数据依赖特性；
2. 掌握两路链式累加算法的基本思想，理解其如何通过减少依赖链长度提升指令级并行潜力；
3. 理解树形归约算法的计算结构，分析其适合并行化实现的原因；
4. 学会使用 `chrono` 进行高精度计时，并通过重复多次测量减少误差；
5. 通过实验数据比较不同算法在串行与并行实现下的实际性能差异，分析理论并行性与实际运行效果之间的关系。

2 实验原理

2.1 问题描述

设数组为 p ，长度为 N ，实验要求计算：

$$S = \sum_{i=0}^{N-1} p[i] \quad (1)$$

其中本实验中令：

$$p[i] = i \quad (2)$$

并取：

$$N = 2^n \quad (3)$$

这样做一方面便于验证结果正确性，另一方面也方便树形归约算法按层次不断折半。

2.2 平凡链式累加算法

最直接的求和方法是逐个元素累加：

$$sum = sum + p[i] \quad (4)$$

该方法结构简单，只需一次线性扫描即可完成全部求和，但它存在严格的循环携带依赖。因为下一次加法必须依赖前一次更新后的 sum ，所以形成了一条长度为 N 的依赖链。在并行视角下，其关键路径长度为 $O(N)$ 。

2.3 两路链式累加与指令级并行

为了降低依赖链长度，可以引入两个独立累加器：

$$sum_1 = sum_1 + p[i], \quad sum_2 = sum_2 + p[i + 1] \quad (5)$$

最后再执行一次：

$$sum = sum_1 + sum_2 \quad (6)$$

这样原本单一的依赖链被拆分为两条较短的独立链。由于 sum_1 与 sum_2 之间互不依赖，处理器有机会同时调度这两条加法指令，从而更好地利用超标量处理器中的多个执行单元。这种优化属于典型的指令级并行优化。

2.4 树形归约算法

树形归约的基本思想是先将相邻两个元素相加，再对得到的中间结果继续两两相加，如此反复，直到只剩下一个结果。其形式可表示为：

$$p[i] = p[2i] + p[2i + 1] \quad (7)$$

每进行一轮，有效数据规模缩小为原来的一半，因此总共只需要：

$$\log_2 N \quad (8)$$

轮归约。与链式算法相比，树形归约将关键路径长度从 $O(N)$ 缩短为 $O(\log N)$ ，理论并行性更强。

2.5 并行树形归约

在树形归约的每一轮中，所有形如

$$p[i] = p[2i] + p[2i + 1] \quad (9)$$

的操作彼此独立，因此可以利用 OpenMP 对每一层内部的循环进行并行化。这样，外层按层次逐轮推进，内层由多个线程同时完成本层的配对求和。

不过，树形归约虽然更适合并行，但这并不意味着它在实际 CPU 上一定快于普通线性求和。其性能还会受到线程调度、同步开销、数组写回开销以及问题规模等因素影响。

:contentReference[oaicite:2]index=2

3 实验平台与实验方案

3.1 实验平台

本实验在 Ubuntu 环境下完成，采用 C++ 编写程序，并使用 OpenMP 实现并行树形归约。实验平台配置如下：

操作系统：Ubuntu

编译器：g++

编译选项：-O2 -fopenmp -std=c++11

并行库：OpenMP

3.2 测试数据设计

为保证结果可复现，本实验采用固定方式初始化数组：

$$p[i] = i \quad (10)$$

此时理论正确结果为：

$$S = 0 + 1 + 2 + \cdots + (N - 1) = \frac{N(N - 1)}{2} \quad (11)$$

该构造方式简单明确，便于人工核对结果。:contentReference[oaicite:3]index=3

3.3 测试规模设计

根据实验要求与内存开销情况，本实验选取如下问题规模进行测试：

1. $n = 15$ ，即 $N = 2^{15} = 32768$
2. $n = 20$ ，即 $N = 2^{20} = 1048576$
3. $n = 24$ ，即 $N = 2^{24} = 16777216$

这样既能够覆盖中小规模与较大规模输入，也能观察随着规模增大，各算法性能变化的趋势。:contentReference[oaicite:4]index=4

3.4 计时方法

实验采用 `chrono::high_resolution_clock` 进行高精度计时，计时单位为纳秒 (ns)。为减少单次测量波动带来的误差，每个算法在同一输入规模下重复执行 10 次，并取平均值作为最终运行时间。

此外，程序使用 `volatile long long sink` 接收函数返回值，以避免编译器在 -O2 优化下将无副作用的计算直接消除，从而保证计时结果具有可信度。:contentReference[oaicite:5]index=5

4 算法设计与代码实现

4.1 算法思路说明

本实验共实现四种求和算法：

1. 平凡算法 `ordinary_alg`：使用单个累加器按顺序逐个累加；
2. 两路链式累加算法 `ilp_alg`：使用两个独立累加器，体现指令级并行思想；
3. 树形归约算法 `optimized_alg`：采用原地两两相加、逐层折半的方式求和；
4. 并行树形归约算法 `parallel_alg`：在树形归约每一层内部使用 OpenMP 并行化。

这四种算法分别对应不同的结构优化思路，因此能够较全面地体现“平凡算法—ILP 优化—树形归约—并行实现”的演进过程。:contentReference[oaicite:6]index=6

4.2 核心代码

下面给出本实验的完整代码。

完整实验代码

```
1 #include<iostream>
2 #include<chrono>
3 #include<vector>
4 #include<omp.h>
5
6 using namespace std;
7 using namespace std::chrono;
8
```

```
9 void init_data(vector<long long>& p) {
10     for (int i = 0; i < p.size(); i++) {
11         p[i] = i;
12     }
13 }
14
15 long long ordinary_alg(vector<long long>& p) {
16     long long sum = 0;
17     for (int i = 0; i < p.size(); i++) {
18         sum += p[i];
19     }
20     return sum;
21 }
22
23 long long ilp_alg(vector<long long>& p) {
24     long long sum1 = 0;
25     long long sum2 = 0;
26     int i = 0;
27     for (; i + 1 < p.size(); i += 2) {
28         sum1 += p[i];
29         sum2 += p[i + 1];
30     }
31     if (i < p.size()) {
32         sum1 += p[i];
33     }
34     return sum1 + sum2;
35 }
36
37 long long optimized_alg(vector<long long>& p) {
38     int m = p.size();
39     for (int n = m; n > 1; n = n / 2) {
40         for (int i = 0; i < n / 2; i++) {
41             p[i] = p[2 * i] + p[2 * i + 1];
42         }
43     }
44     return p[0];
45 }
46
47 long long parallel_alg(vector<long long>& p) {
```

```
48     int m = p.size();
49     for (int n = m; n > 1; n = n / 2) {
50 #pragma omp parallel for
51         for (int i = 0; i < n / 2; i++) {
52             p[i] = p[2 * i] + p[2 * i + 1];
53         }
54     }
55     return p[0];
56 }
57
58 int main() {
59     int n;
60     cout << "please cin the amount of n" << endl;
61     cin >> n;
62
63     int s = 1 << n;
64     vector<long long> p(s, -1);
65     init_data(p);
66
67     volatile long long sink = 0;
68
69     long long total1 = 0;
70     long long total2 = 0;
71     long long total3 = 0;
72     long long total4 = 0;
73
74     int r = 10;
75
76     for (int i = 0; i < r; i++) {
77         auto start1 = high_resolution_clock::now();
78         sink = ordinary_alg(p);
79         auto end1 = high_resolution_clock::now();
80         total1 += duration_cast<nanoseconds>(end1 - start1).count();
81
82         auto start2 = high_resolution_clock::now();
83         sink = ilp_alg(p);
84         auto end2 = high_resolution_clock::now();
85         total2 += duration_cast<nanoseconds>(end2 - start2).count();
86     }
```

```
87     vector<long long> temp = p;
88     auto start3 = high_resolution_clock::now();
89     sink = optimized_alg(temp);
90     auto end3 = high_resolution_clock::now();
91     total3 += duration_cast<nanoseconds>(end3 - start3).count();
92
93     vector<long long> temp2 = p;
94     auto start4 = high_resolution_clock::now();
95     sink = parallel_alg(temp2);
96     auto end4 = high_resolution_clock::now();
97     total4 += duration_cast<nanoseconds>(end4 - start4).count();
98 }
99
100 double average_time1 = 1.0 * total1 / r;
101 double average_time2 = 1.0 * total2 / r;
102 double average_time3 = 1.0 * total3 / r;
103 double average_time4 = 1.0 * total4 / r;
104
105 cout << "the answer of ordinary_alg is: " << ordinary_alg(p) << endl;
106 cout << "the time cost of ordinary_alg is: " << average_time1 << endl;
107
108 cout << "the answer of ilp_alg is: " << ilp_alg(p) << endl;
109 cout << "the time cost of ilp_alg is: " << average_time2 << endl;
110
111 vector<long long> temp = p;
112 cout << "the answer of optimized_alg is: " << optimized_alg(temp) << endl;
113 cout << "the time cost of optimized_alg is: " << average_time3 << endl;
114
115 vector<long long> temp2 = p;
116 cout << "the answer of parallel_alg is: " << parallel_alg(temp2) << endl;
117 cout << "the time cost of parallel_alg is: " << average_time4 << endl;
118
119 cout << "speed up of ilp_alg: " << average_time1 / average_time2 << endl;
120 cout << "speed up of optimized_alg: " << average_time1 / average_time3 <<
    endl;
121 cout << "speed up of parallel_alg: " << average_time1 / average_time4 << endl
    ;
122
123 return 0;
```

124 }

4.3 代码结构与关键实现解析

4.3.1 数据初始化

```
1 void init_data(vector<long long>& p) {  
2     for (int i = 0; i < p.size(); i++) {  
3         p[i] = i;  
4     }  
5 }
```

该函数用于生成测试数据，将数组初始化为 $p[i] = i$ 。由于数据分布规则明确，便于验证结果正确性，并能避免随机数据引入的额外不确定性。

4.3.2 平凡算法

```
1 long long ordinary_alg(vector<long long>& p) {  
2     long long sum = 0;  
3     for (int i = 0; i < p.size(); i++) {  
4         sum += p[i];  
5     }  
6     return sum;  
7 }
```

平凡算法使用单个累加器依次累加所有元素。其优点是代码简单、访存连续、局部变量可保留在寄存器中；其缺点是存在严格的循环依赖，不利于指令级并行。

4.3.3 两路链式累加算法

```
1 long long ilp_alg(vector<long long>& p) {  
2     long long sum1 = 0;  
3     long long sum2 = 0;  
4     int i = 0;  
5     for (; i + 1 < p.size(); i += 2) {  
6         sum1 += p[i];  
7         sum2 += p[i + 1];  
8     }  
9     if (i < p.size()) {  
10        sum1 += p[i];  
11    }
```

```
11     }
12     return sum1 + sum2;
13 }
```

该算法引入两个独立累加器 `sum1` 和 `sum2`，从而将原本的一条长依赖链拆成两条较短的依赖链。这种结构更适合超标量处理器利用多个执行单元进行指令调度，体现了指令级并行思想。

4.3.4 树形归约算法

```
1 long long optimized_alg(vector<long long>& p) {
2     int m = p.size();
3     for (int n = m; n > 1; n = n / 2) {
4         for (int i = 0; i < n / 2; i++) {
5             p[i] = p[2 * i] + p[2 * i + 1];
6         }
7     }
8     return p[0];
9 }
```

树形归约算法按层次将相邻两个元素合并，并将结果原地写回数组前半部分。该算法理论并行性较强，但其串行实现需要多轮循环，并且每轮都涉及数组写回，因此常数开销较大。

4.3.5 并行树形归约算法

```
1 long long parallel_alg(vector<long long>& p) {
2     int m = p.size();
3     for (int n = m; n > 1; n = n / 2) {
4 #pragma omp parallel for
5         for (int i = 0; i < n / 2; i++) {
6             p[i] = p[2 * i] + p[2 * i + 1];
7         }
8     }
9     return p[0];
10 }
```

并行树形归约算法利用 OpenMP 对每一轮内部的归约操作进行并行调度。由于同一层的各个配对求和任务互不依赖，因此适合并行执行。但线程创建、调度与同步也会带来额外开销，尤其在问题规模较小时影响更明显。

4.3.6 计时逻辑

```

1  for (int i = 0; i < r; i++) {
2      auto start1 = high_resolution_clock::now();
3      sink = ordinary_alg(p);
4      auto end1 = high_resolution_clock::now();
5      total1 += duration_cast<nanoseconds>(end1 - start1).count();
6
7      auto start2 = high_resolution_clock::now();
8      sink = ilp_alg(p);
9      auto end2 = high_resolution_clock::now();
10     total2 += duration_cast<nanoseconds>(end2 - start2).count();
11
12     vector<long long> temp = p;
13     auto start3 = high_resolution_clock::now();
14     sink = optimized_alg(temp);
15     auto end3 = high_resolution_clock::now();
16     total3 += duration_cast<nanoseconds>(end3 - start3).count();
17
18     vector<long long> temp2 = p;
19     auto start4 = high_resolution_clock::now();
20     sink = parallel_alg(temp2);
21     auto end4 = high_resolution_clock::now();
22     total4 += duration_cast<nanoseconds>(end4 - start4).count();
23 }

```

程序对每种算法重复执行 10 次并统计总耗时，最终求平均值。由于 `optimized_alg` 和 `parallel_alg` 会修改数组，因此在调用前先复制一份原数组，以保证每次测试的输入一致。:contentReference[oaicite:7]index=7

4.4 伪代码描述

Algorithm 1 n 个数求和的四种实现方式

Input: 数组 p

Output: 数组元素总和

```

1: function ORDINARYALG( $p$ )
2:    $sum \leftarrow 0$ 
3:   for  $i = 0$  to  $N - 1$  do
4:      $sum \leftarrow sum + p[i]$ 
5:   end for

```

```

6:   return  $sum$ 
7: end function
8: function ILPALG( $p$ )
9:    $sum_1 \leftarrow 0, sum_2 \leftarrow 0$ 
10:  for  $i = 0$  to  $N - 2$  step 2 do
11:     $sum_1 \leftarrow sum_1 + p[i]$ 
12:     $sum_2 \leftarrow sum_2 + p[i + 1]$ 
13:  end for
14:  if  $N$  为奇数 then
15:     $sum_1 \leftarrow sum_1 + p[N - 1]$ 
16:  end if
17:  return  $sum_1 + sum_2$ 
18: end function
19: function OPTIMIZEDALG( $p$ )
20:   $n \leftarrow N$ 
21:  while  $n > 1$  do
22:    for  $i = 0$  to  $n/2 - 1$  do
23:       $p[i] \leftarrow p[2i] + p[2i + 1]$ 
24:    end for
25:     $n \leftarrow n/2$ 
26:  end while
27:  return  $p[0]$ 
28: end function
29: function PARALLELALG( $p$ )
30:   $n \leftarrow N$ 
31:  while  $n > 1$  do
32:    并行执行以下循环
33:    for  $i = 0$  to  $n/2 - 1$  do
34:       $p[i] \leftarrow p[2i] + p[2i + 1]$ 
35:    end for
36:     $n \leftarrow n/2$ 
37:  end while
38:  return  $p[0]$ 
39: end function

```

5 实验结果

在当前实验平台下，得到如下测试结果。每组数据均重复运行 10 次并取平均值。

n	ordinary_alg(ns)	ilp_alg(ns)	optimized_alg(ns)	parallel_alg(ns)
5	111.2	152.3	169.4	1.41967×10^4
10	936.7	693.4	1512.9	2.41351×10^4
15	1.97160×10^4	1.69047×10^4	4.03892×10^4	6.85470×10^4
20	1.43018×10^6	9.81512×10^5	3.62507×10^6	3.19072×10^6
25	8.49219×10^7	2.77975×10^7	2.03020×10^8	8.23004×10^7

表 1: 不同规模下各算法平均运行时间

根据程序输出可进一步得到加速比：

n	ilp_alg 加速比	optimized_alg 加速比	parallel_alg 加速比
5	1.177	1.059	0.012
10	1.350	0.619	0.038
15	1.166	0.488	0.287
20	1.457	0.394	0.448
25	3.055	0.418	1.031

表 2: 相对于 ordinary_alg 的加速比

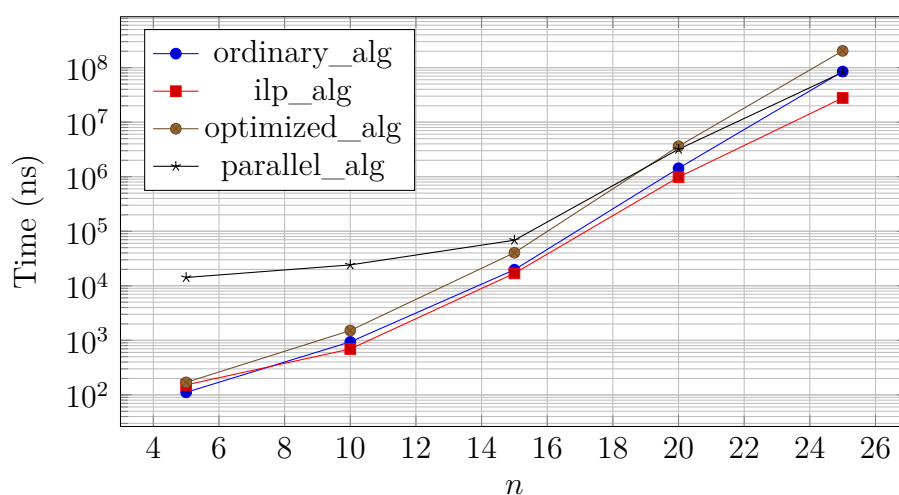


图 1: 不同算法运行时间对比

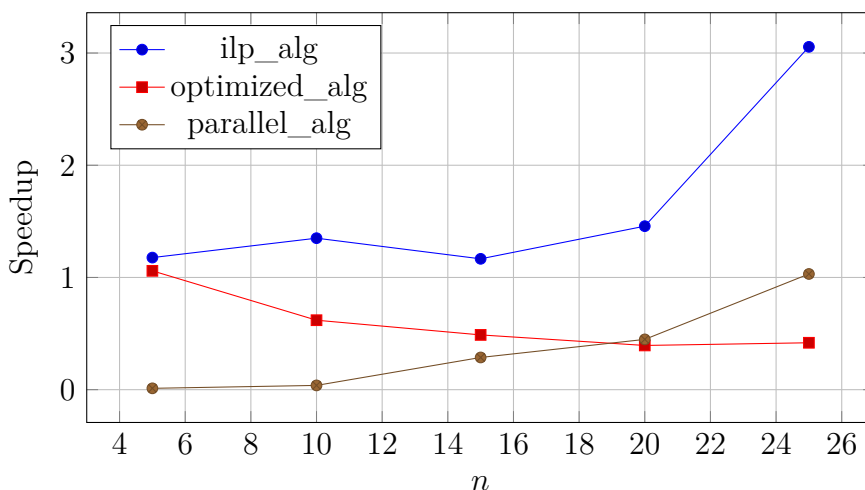


图 2: 不同优化算法加速比变化

从实验结果可以看出，不同算法在不同问题规模下表现差异明显。两路链式累加算法在各组测试中均优于平凡算法，且随着规模增大优势更加明显；树形归约串行实现始终慢于平凡算法；OpenMP 并行树形归约在小规模下开销很大，在较大规模下性能逐渐改善，并在 $n = 25$ 时略微超过 ordinary_alg。

6 实验过程中遇到的问题与解决方法

在实验过程中，先后遇到了若干典型问题，这些问题本身也帮助加深了对算法与体系结构关系的理解。

6.1 树形归约结果出错

最初实现树形归约时，数组类型使用的是 `int`。当问题规模增大时，中间归约结果已经远远超过 `int` 的取值范围，导致程序输出负值或错误结果。后来将数组类型改为 `long long`，问题得到解决。这说明在并行或归约问题中，不仅最终结果范围要考虑，中间结果范围同样不能忽视。

6.2 对“优化”概念的理解偏差

实验初期容易误认为只要算法在理论上更适合并行，其串行实现也应当更快。但实际测试表明并非如此。树形归约优化的重点是“关键路径更短、并行潜力更大”，而 ordinary_alg 优化的是“串行执行路径极简、常数开销极小”。这两种“优化”面向的是不同层面的性能目标，不能简单混为一谈。

7 结论

本实验围绕 n 个数求和问题，实现并比较了平凡链式累加、两路链式累加、树形归约以及 OpenMP 并行树形归约四种算法。结合理论分析与实验结果，可以得到以下结论：

1. 平凡链式累加算法结构简单、访存连续，在小规模和中等规模下具有较好的基线性能；
2. 两路链式累加算法通过引入两个独立累加器，降低了依赖链长度，较好地体现了指令级并行优化思想，并且在本实验所有测试规模下都优于平凡算法，是本次实验中综合表现最好的优化方法；
3. 树形归约算法虽然理论上将关键路径由 $O(N)$ 缩短为 $O(\log N)$ ，具有较强的并行潜力，但其串行实现由于多轮循环、索引计算和数组写回，实际开销较大，在本实验中始终慢于平凡算法；
4. OpenMP 并行树形归约在小规模下受到线程调度与同步开销影响，性能明显不佳；随着问题规模增大，其性能逐渐提升，并在 $n = 25$ 时略微超过平凡算法，说明树形归约的并行优势需要足够大的问题规模才能体现；
5. 程序性能不仅取决于算法的渐进复杂度，还受到数据依赖、访存方式、编译器优化以及并行实现开销的共同影响。

综上，本实验较系统地展示了从链式累加到指令级并行，再到树形归约和并行归约的算法演进过程。实验结果表明，在当前 CPU 平台上，两路链式累加这种面向单核执行的指令级并行优化最有效；而树形归约的优势更多体现在大规模并行执行场景中，而不是简单的串行实现中。这说明在实际程序优化中，需要根据硬件平台和问题规模选择合适的优化策略，而不能仅依据理论并行性判断算法优劣。